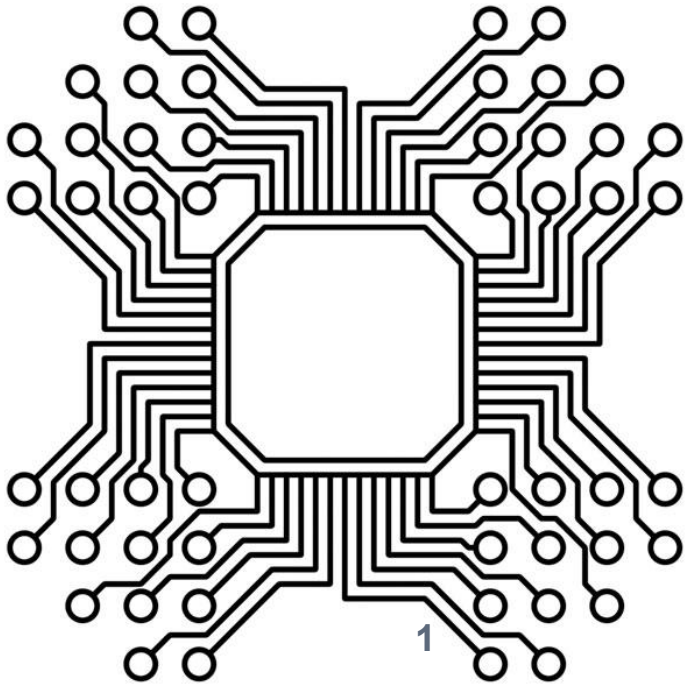
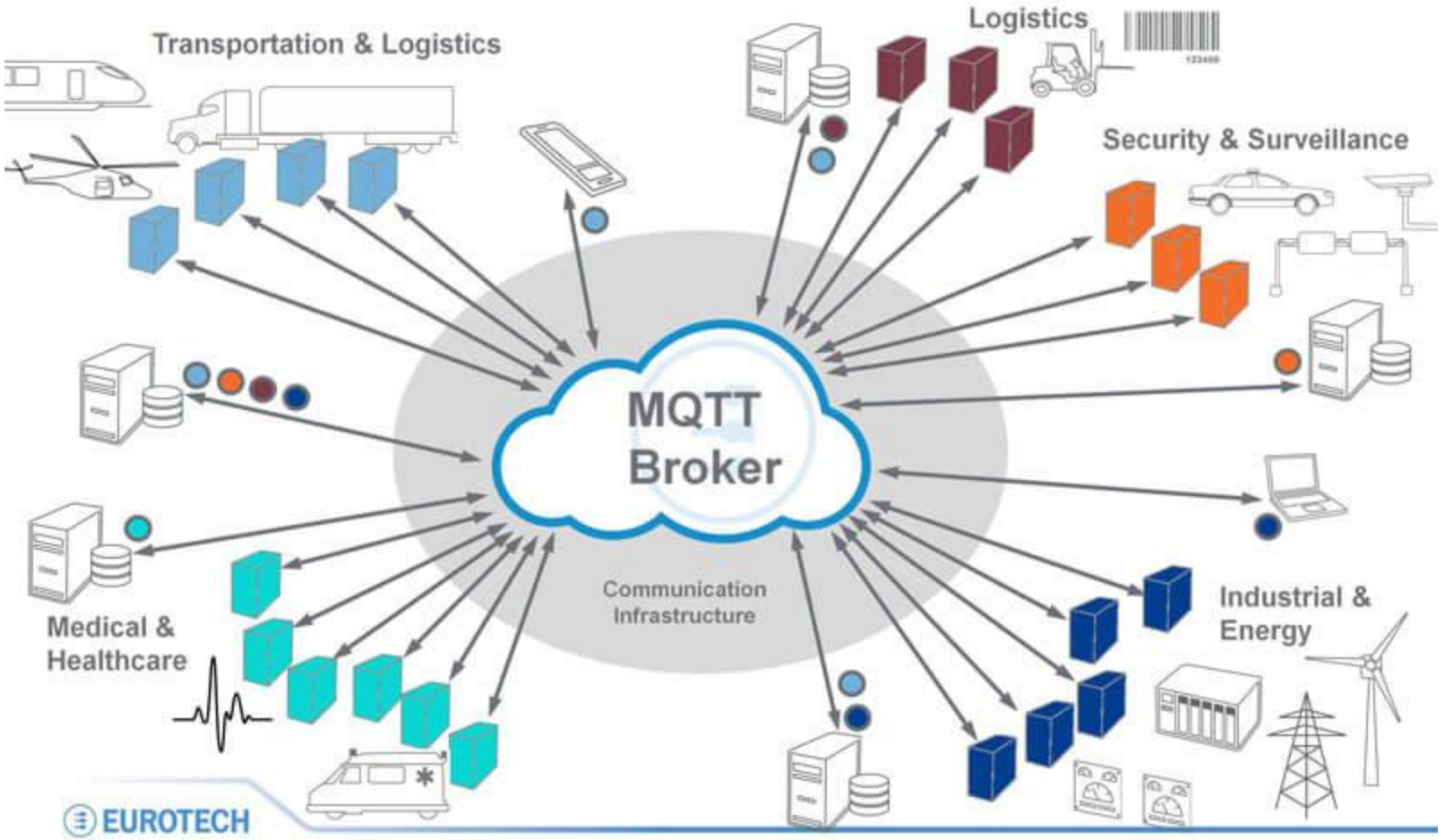
MQTT PROTOCOL

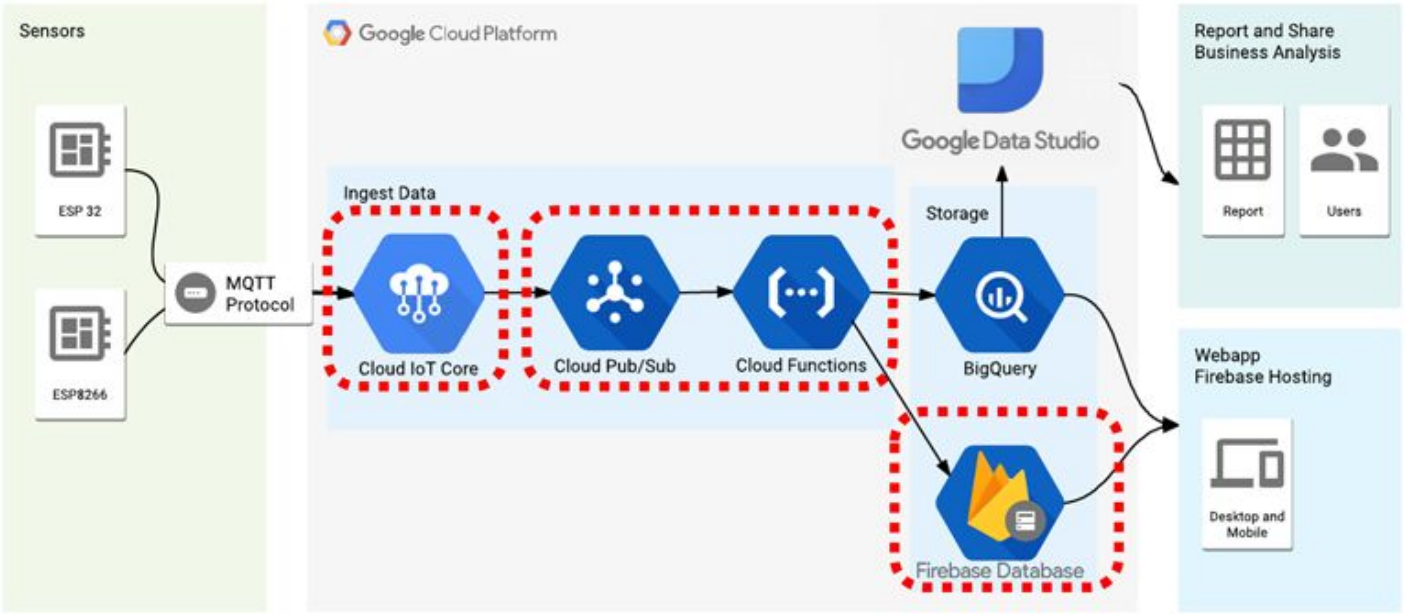
Lecturer: Dr. Bui Ha Duc
Dept. of Mechatronics
Email: ducbh@hcmute.edu.vn

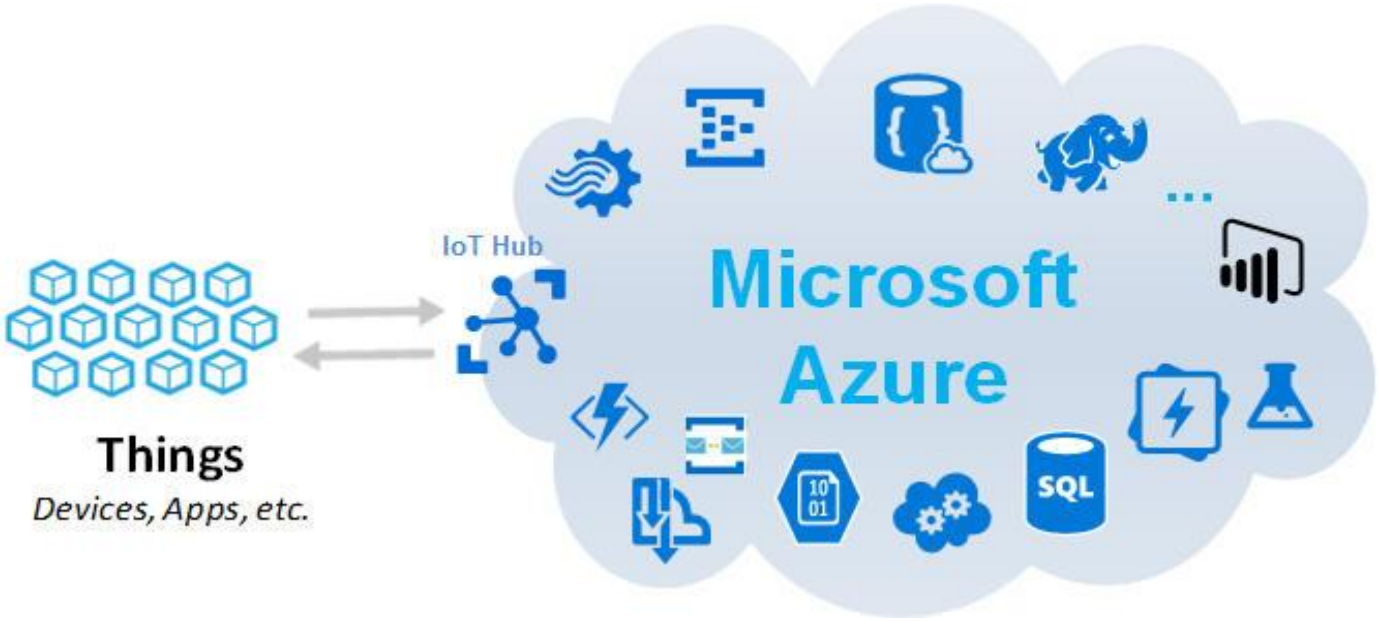


The Internet of Things

Decoupling Producers & Consumers of M2M Device Data




Sensors
ESP 32
ESP8266
MQTT Protocol
Google Cloud Platform
Ingest Data
Cloud IoT Core
Cloud Pub/Sub
Cloud Functions
Storage
BigQuery
Google Data Studio
Firebase Database
Report and Share Business Analysis
Report
Users
Webapp Firebase Hosting
Desktop and Mobile

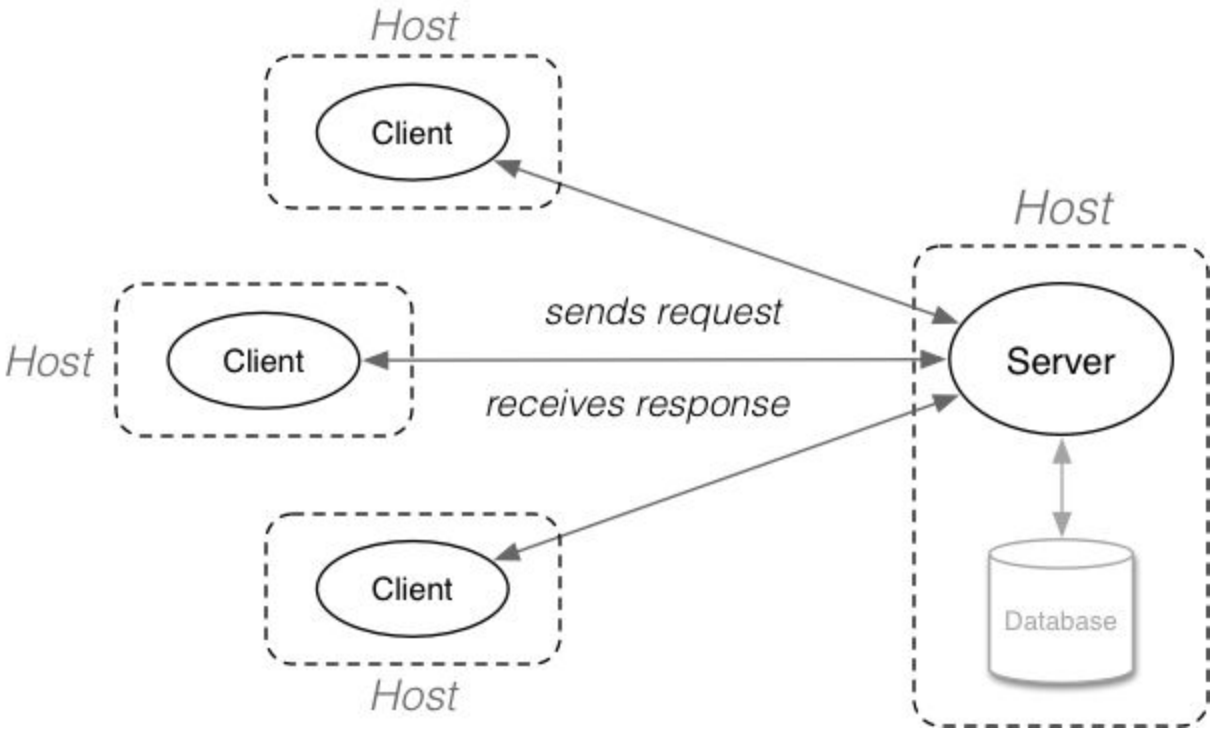

IoT Hub
Microsoft Azure
Things
Devices, Apps, etc.
SQL

Introduction to MQTT

- MQTT - Message Queuing Telemetry Transport
- MQTT was invented in 1999 by IBM as a Client Server publish/subscribe messaging transport protocol
- Advantages of MQTT:
  - Lightweight
  - Simple
  - Easy to implement
- MQTT is faster than HTTP's.
- IBM used it internally, only release after version 3.1 in 2012.
- The current version is MQTT 5, released in 2019

Client-Server Model

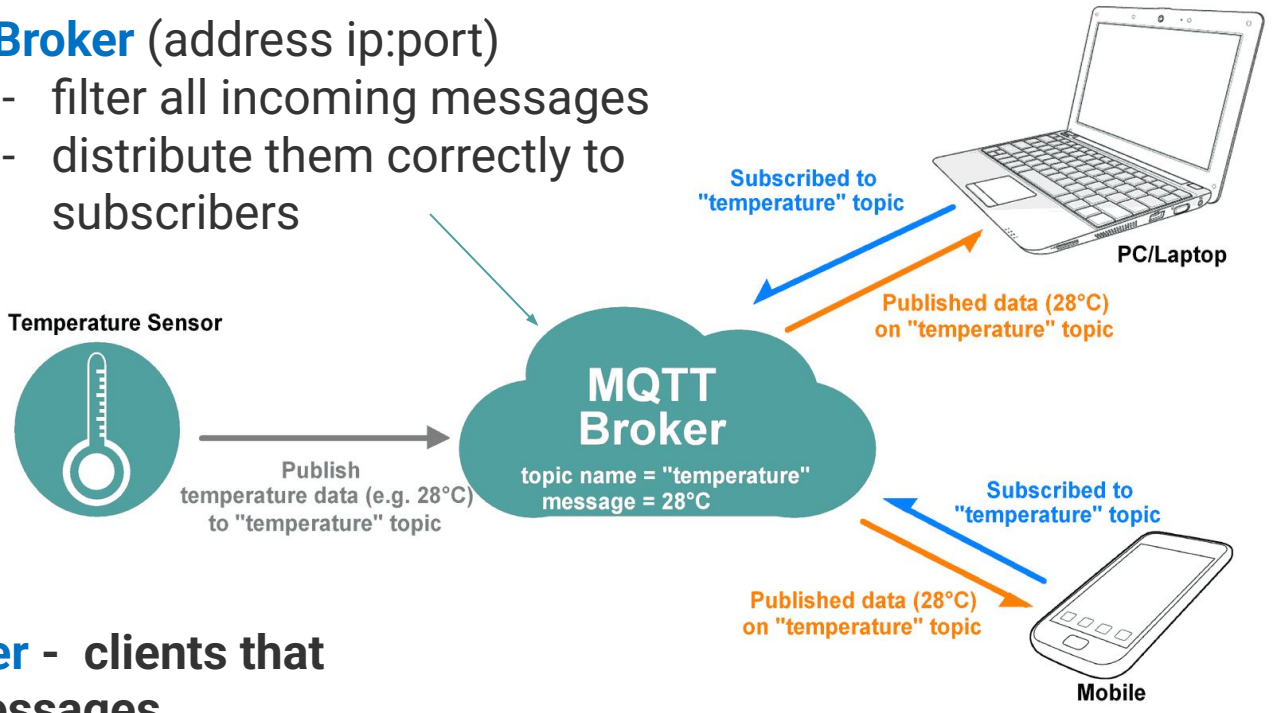
- Traditional Client-Server architechture



Publish/subscribe Model

- Publish/Subscribe Model

Broker (address ip:port)
- filter all incoming messages
- distribute them correctly to subscribers



Publisher - **clients that send messages**

never contact each other directly

Subscribers - **clients that receive the messages**

Publish/subscribe Model

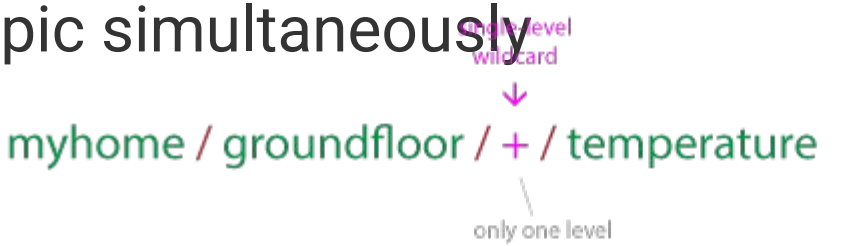
- Publish/subscribe model has 3 features
  - Publisher and subscriber do not need to know each other (for example, no exchange of IP address and port).
  - Publisher and subscriber do not need to run at the same time.
  - Operations on both components do not need to be interrupted during publishing or receiving.
- MQTT uses **topic** filtering of messages
  - Topic is an UTF-8 string that the broker uses to filter messages for each connected client
  - Topic filters can contain wildcards: +, #

MQTT Topic

- Topic consists of one or more topic levels, separated by "/"



- A subscriber can use wild card to subscribe to multiple topic simultaneously



- ✓ myhome / groundfloor / livingroom / temperature
- ✓ myhome / groundfloor / kitchen / temperature
- ✗ myhome / groundfloor / kitchen / brightness
- ✗ myhome / firstfloor / kitchen / temperature
- ✗ myhome / groundfloor / kitchen / fridge / temperature



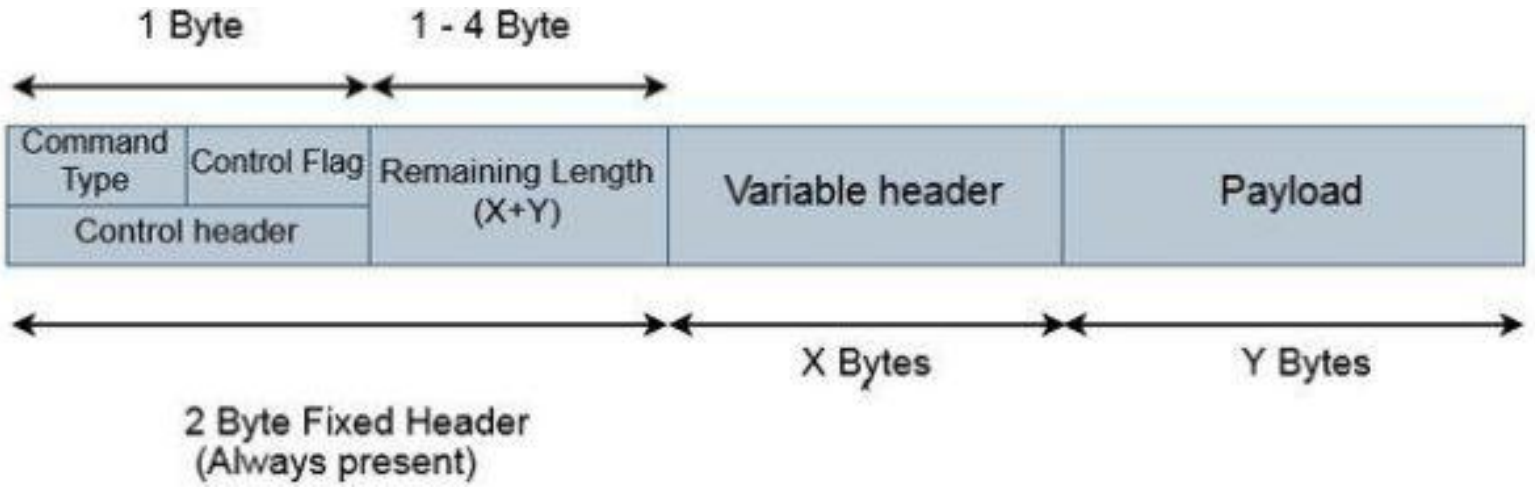
- ✓ myhome / groundfloor / livingroom / temperature
- ✓ myhome / groundfloor / kitchen / temperature
- ✓ myhome / groundfloor / kitchen / brightness
- ✗ myhome / firstfloor / kitchen / temperature

MQTT Message

- MQTT only has several types of message:
  - **CONNECT**: Used for clients to send connection requests to the broker
  - **PUBLISH**: Used by the client to publish messages to the broker
  - **(UN)SUBSCRIBE**: Used by the client to receive/stop receiving messages from the broker
  - **ACKNOWLEDGEMENT**: Used by receiver to confirm an action
    - CONNACK
    - PUBACK
    - PUBREC
    - SUBACK
    - UNSUBACK

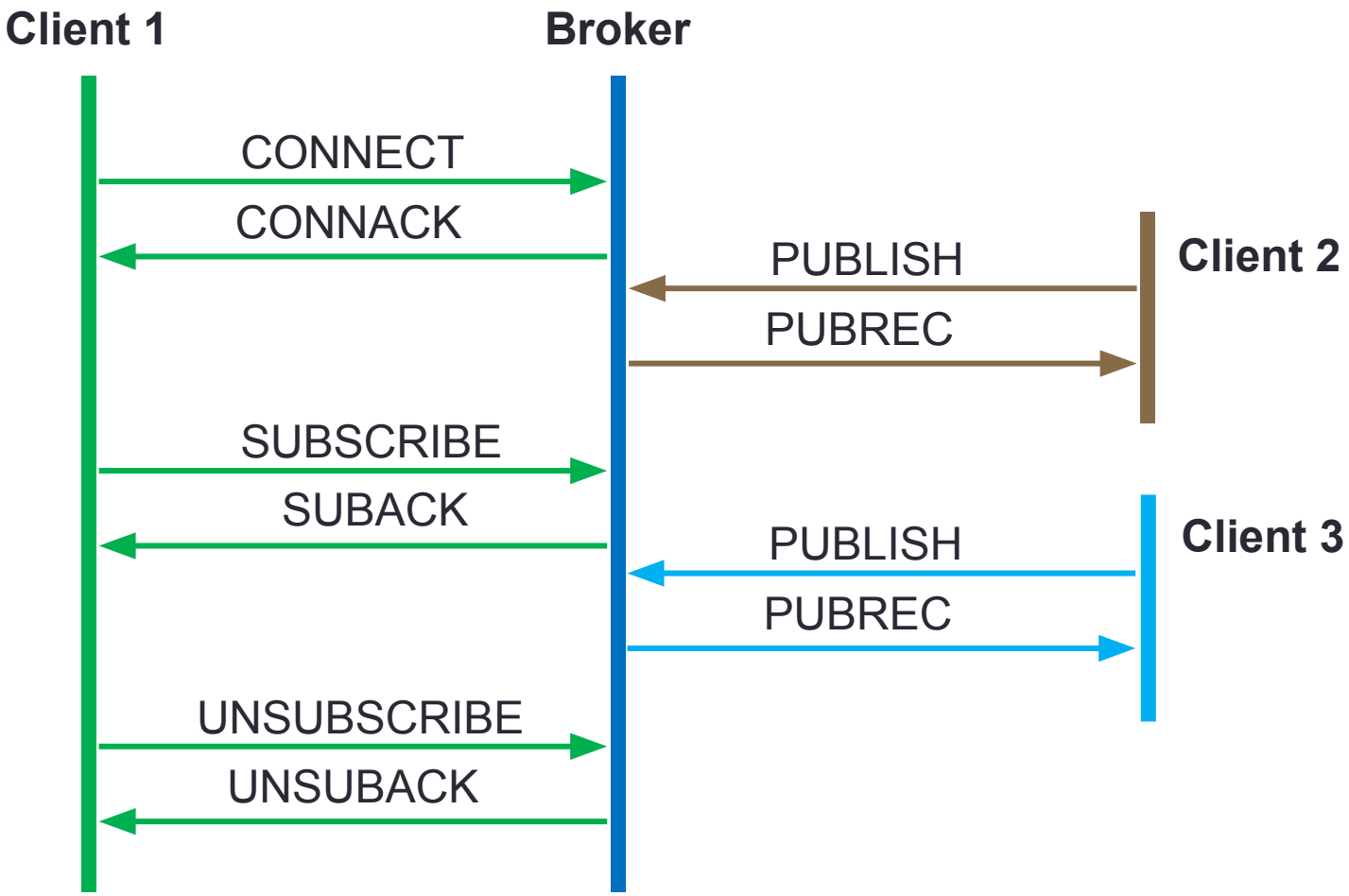
MQTT Message

- MQTT Message format:



(refer to this link for more detail: https://openlabpro.com/guide/mqtt-packet-format/)

MQTT Message flow



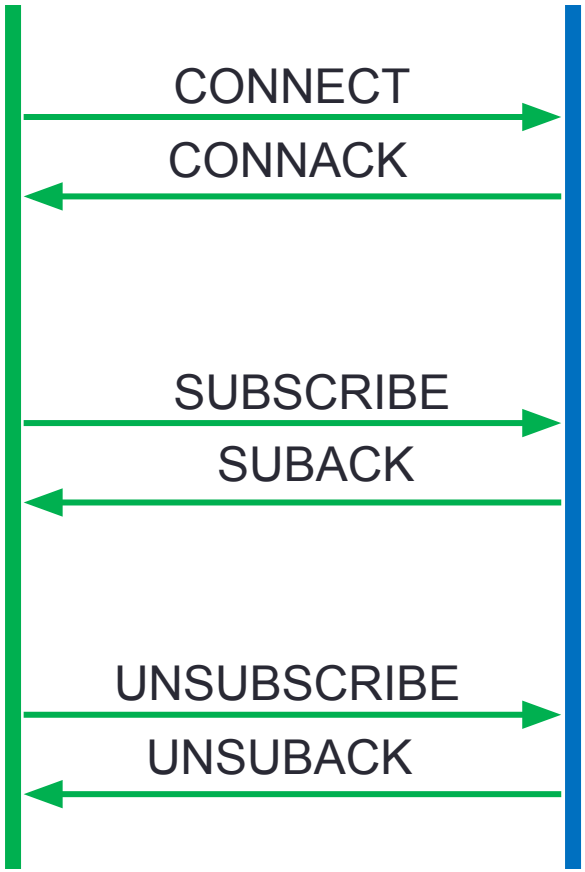
MQTT is a command protocol, each command will be response by the receiver

MQTT Message flow - CONNECT

Connect message is sent by the client to the broker to initiate a connection

MQTT-Packet:
CONNECT

contains:	Example
clientId	"client-1"
cleanSession	true
username (optional)	"hans"
password (optional)	"letmein"
lastWillTopic (optional)	"/hans/will"
lastWillQos (optional)	2
lastWillMessage (optional)	"unexpected exit"
lastWillRetain (optional)	false
keepAlive	60



Server to client, connect acknowledgment

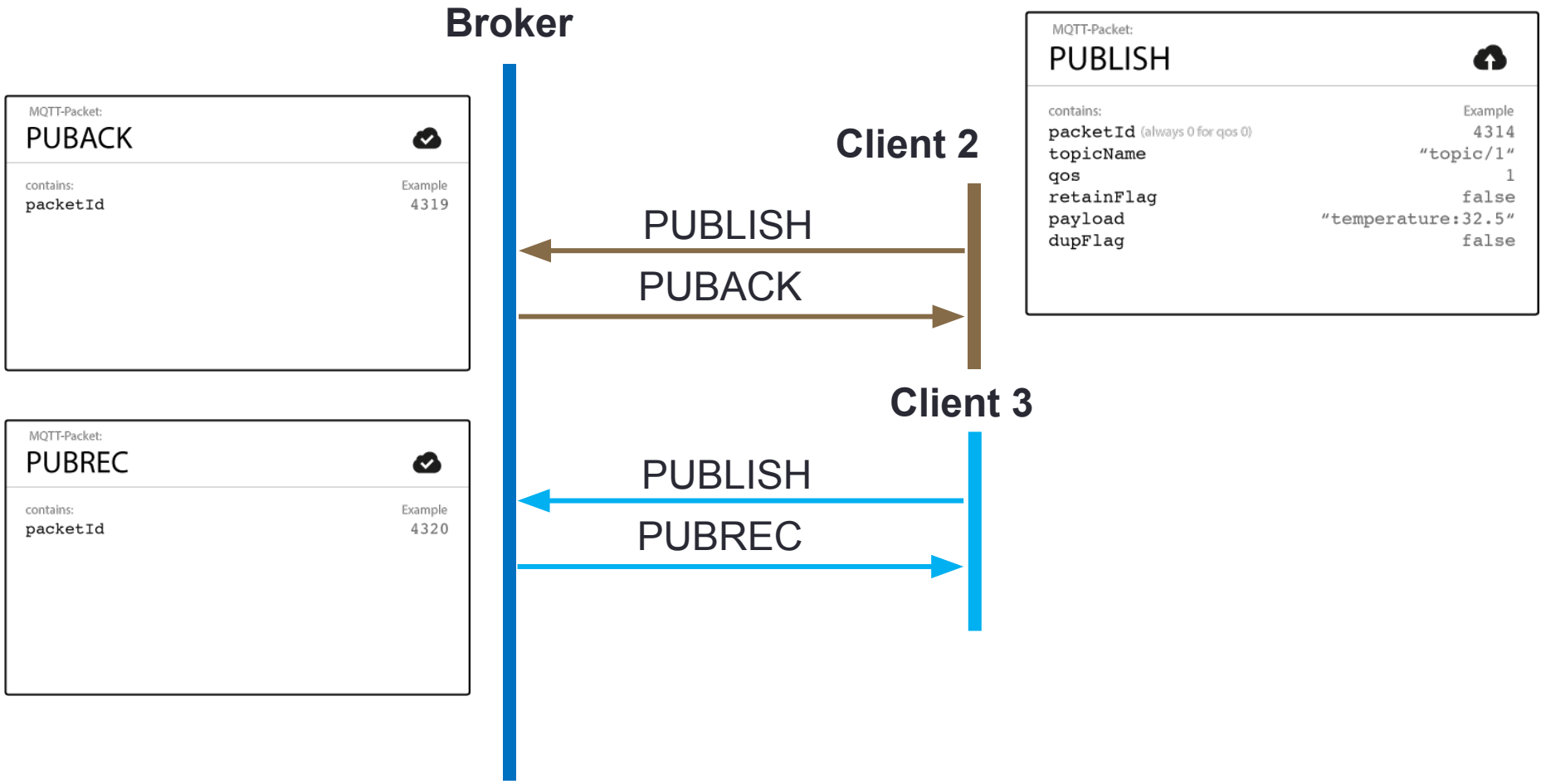
MQTT-Packet:
CONNACK

contains:	Example
sessionPresent	true
returnCode	0

Return Code	Return Code Response
0	Connection accepted
1	Connection refused, unacceptable protocol version
2	Connection refused, identifier rejected
3	Connection refused, server unavailable
4	Connection refused, bad user name or password
5	Connection refused, not authorized

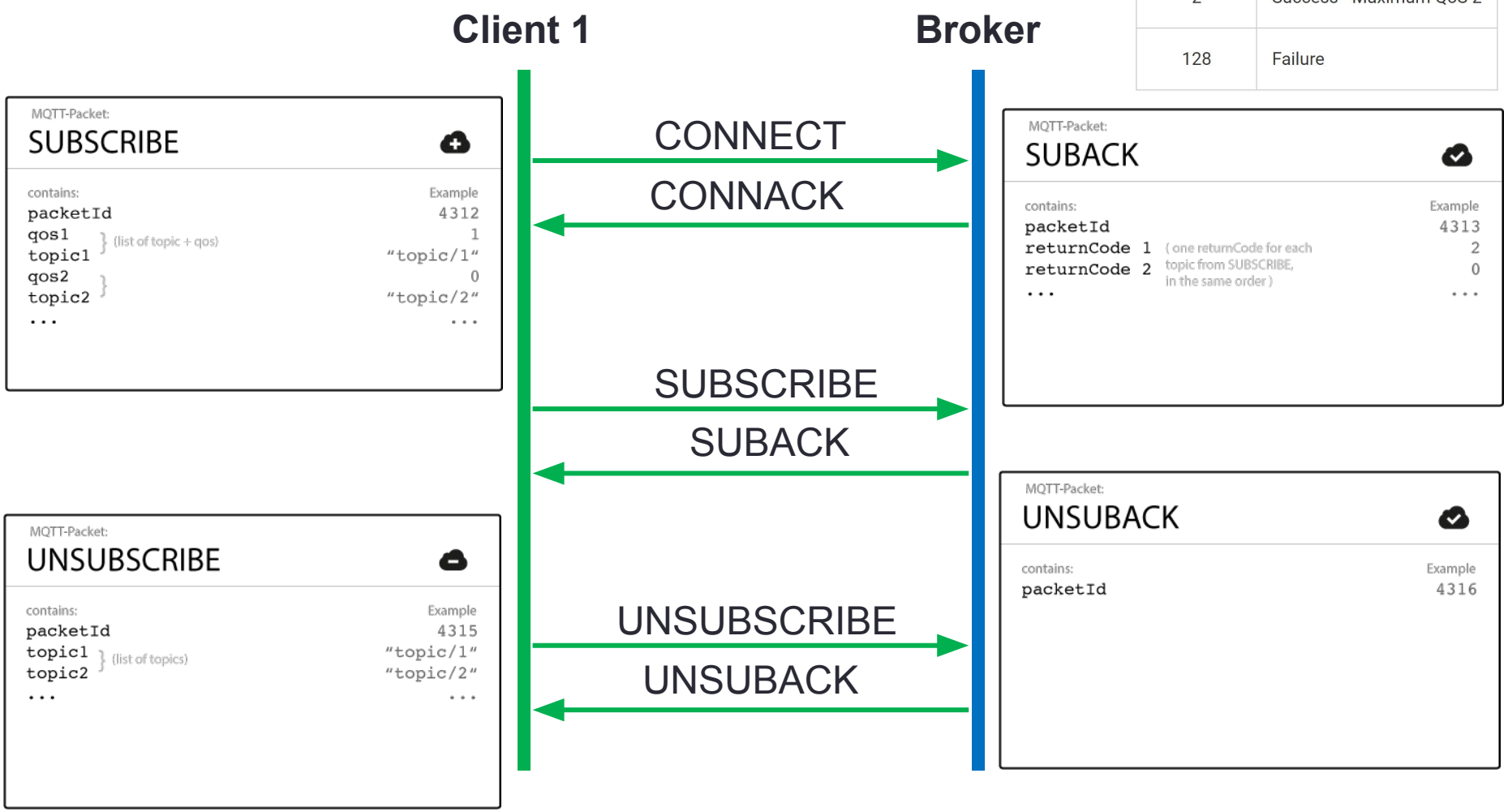
(refer to this link for more detail: https://www.hivemq.com/blog/mqtt-essentials-part-3-client-broker-connection-establishment/)

MQTT Message flow - PUBLISH

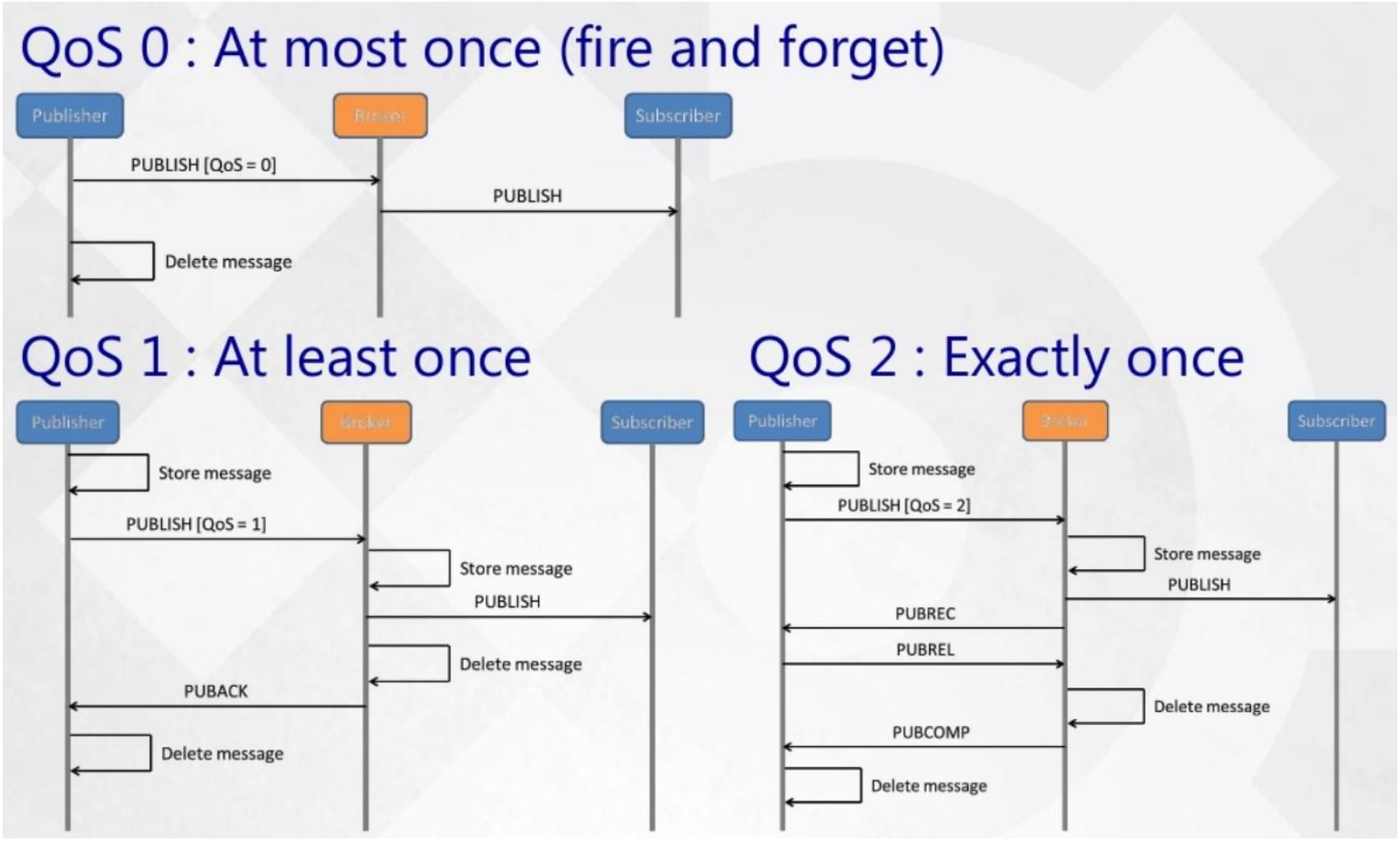


MQTT Message flow

Return Code	Return Code Response
0	Success - Maximum QoS 0
1	Success - Maximum QoS 1
2	Success - Maximum QoS 2
128	Failure



QoS – Quality of service



Setup MQTT Platform

- MQTT Broker:
  - Online Broker:
    - Paid: Amazon Azure, Google Cloud, IBM Cloud
    - “Free”: Eclipse, Mosquitto, Hivemq, emqx.io, Flespi…
  - On PC Broker: Mosquitto
    - https://mosquitto.org/

- MQTT Client library:
  - Eclipse Paho - https://www.eclipse.org/paho/
  - Notes: compile code C with “-lpaho-mqtt3c”

  e.g. gcc pub.c -o pub -lpaho-mqtt3c

Install Paho-MQTT

- sudo apt update
- git clone https://github.com/eclipse/paho.mqtt.c.git
- cd paho.mqtt.c
- sudo apt install libssl-dev
- make
- sudo make install